

3L 前端架构迁移方案

版本：v1.0 | 日期：2026-05-26 | 状态：草案

1. 现状与问题

当前架构

```
frontend/
├─ *.html          # 15 个静态 HTML 页面
├─ public/js/pages/ # 15 个对应的 vanilla JS 文件
├─ css/pages/       # 15 个对应的 CSS 文件
├─ dist/            # Vite 构建产物 (CSS 压缩 + 前缀)
├─ build.py         # 构建脚本
├─ vite.config.js   # Vite 配置 (多页模式, 仅处理 CSS)
└─ package.json     # 仅 vite + pinyin-pro
```

核心问题：

#	问题	影响
1	数据变更后页面不会自动重绘	用户改趋势列表 → 页面不变，需要手动刷新/重新生成
2	JS 全局作用域，无组件化	CSS 类名冲突需手动排查 (stock_card.js 的 chart-container 被 monitor.css 覆盖过)
3	状态管理靠 DOM + 全局变量	一个页面改数据，另一个页面不知道
4	15 个 HTML + 15 个 JS + 15 个 CSS 独立文件	公共逻辑复制粘贴 (stock_card.js、nav.js 每个页面单独引用)

2. 目标架构

```
frontend/
├─ src/
│  ├─ App.tsx          # 路由入口
│  ├─ pages/
│  │  ├─ Monitor.tsx   # 盯盘 (首期迁移)
│  │  ├─ Workbench.tsx # 工作台
│  │  ├─ Review.tsx    # 复盘
│  │  └─ ...           # 逐步迁移
│  ├─ components/
│  │  ├─ StockCard.tsx # 个股卡片 (复用最广)
│  │  ├─ Nav.tsx       # 导航栏
│  │  ├─ KLineChart.tsx # K 线图
│  │  └─ ...
│  ├─ hooks/
│  │  └─ useApi.ts     # 通用 API 请求
```

```
| | └─ useReviewData.ts # 复盘数据
| └─ api/
|   └─ index.ts          # 所有 API 封装
|   └─ types/
|     └─ index.ts        # 类型定义
└─ index.html            # 唯一入口 (SPA 模式)
└─ vite.config.ts        # Vite + React 配置
└─ tsconfig.json
└─ package.json          # React + TypeScript + Vite
```

架构决策

决策	选择	理由
框架	React + TypeScript	用户需求：数据变 → 组件自动重绘
构建	Vite (已有, 升级配置)	Vite 已在用, 加 React 插件即可
路由	React Router (单页应用)	多页模式下组件无法共享状态
样式	CSS Modules	避免全局类名冲突, 逐个迁移
状态管理	React Context + useApi hooks	目前不需要 Redux, 数据量小
图表	SVG React 组件	后端生成的 SVG 直接内嵌, 无需额外库

3. 迁移策略：渐进式替换

核心原则：一次迁一页，两边共存

新 (React) 和旧 (静态 HTML) 同时在线，不走 big bang。

```
后端 server.py:
/monitor      → React 版 monitor.html 新
/review       → 旧 review.html        逐步替换
/watchlist    → 旧 watchlist.html     逐步替换
...
```

迁移顺序

阶段	页面	原因	复杂度
Phase 1	监控(monitor) + 个股卡片(stock_card)	复用最广，组件化收益最大	☆☆
Phase 2	复盘(review) + 工作台(journal)	核心流程页，数据流复杂	☆☆☆
Phase 3	自选股(watchlist) + 趋势(trend_candidates)	交互复杂 (搜索/排序/勾选)	☆☆☆
Phase 4	其余页面 (涨幅/宏观/行业/tips/模拟/技巧)	数据展示为主，迁移快	☆

4. 数据流设计



关键：后端 API 一条不改。 所有改进在前端层面。

当前所有 API 端点：

API	用途	迁移后
/api/review/get	复盘数据	React 组件 fetch
/api/review/dates	复盘日期列表	React 组件 fetch
/api/review/{date}	历史复盘	React 组件 fetch
/api/trend/candidates	趋势候选	React 组件 fetch
/api/trend/tracked	已跟踪趋势	React 组件 fetch
/api/watchlist/...	自选股操作	React 组件 fetch
/api/monitor/...	盯盘数据	React 组件 fetch
/api/stock/...	个股分析	React 组件 fetch

5. 构建与部署

项目	说明
开发	<code>npm run dev</code> → Vite dev server (port 5173) → proxy API 到 8080
构建	<code>npm run build</code> → Vite 编译 TypeScript + 打包 → dist/
部署	<code>python3 build.py</code> → 复制 dist/ 到前端目录 → <code>systemctl restart 3l-server</code>
共存	旧 HTML 文件仍保留，server.py 通过路由前缀分发新旧版本

build.py 变更： 简化。不再需要 inject_stock_card_css 等补丁，因为 React 的 CSS Modules 自带作用域隔离。

6. 风险与注意事项

- 后端不动** — 这是规则。迁移期间不改后端 API，确保新旧前端都能用
- 不重写逻辑** — 从现有 JS 直接翻译成 React，不做功能改进。改进是另一个话题
- SVG 图表** — 后端生成的 SVG 用 `dangerouslySetInnerHTML` 内嵌，不需要重写
- 旧页面保留** — 出问题能快速回退：改 server.py 路由指向旧 HTML 即可

5. TypeScript 严格度 — 先 loose (`strict: false`)，稳定后再收严

7. 执行计划

Phase 0.5: 清理冗余文件（1 次提交）

React 迁移前先把项目根的无用旧文件清理掉，避免混淆。

```
步骤 1: 确认哪些文件是活的
server.py 的 FE_DIR = frontend/dist/ (优先)
回退路径 = 项目根目录 (只有 dist 里没有时才用)

步骤 2: 检查项目根 14 个 .html 是否被任何地方引用
grep -r "\.html" server.py backend/ 确认引用关系

步骤 3: 删除确认无用的旧文件
项目根目录的 *.html (确认无引用后)
项目根目录的 CSS/JS (如果存在)
旧 build.py 中的 inject_stock_card_css 等补丁 (迁移后不再需要)

步骤 4: 验证
systemctl restart 3l-server → 页面正常打开
所有 API 端点正常返回

测试:
手动打开 monitor/review/workbench 等页面, 确认正常渲染
pytest tests/ 全回归
```

Phase 0: 基建搭建（1 次提交）

```
步骤 1: 安装依赖
npm install react react-dom react-router-dom
npm install -D @types/react @types/react-dom @vitejs/plugin-react typescript

步骤 2: Vite 配置 + TypeScript 配置
vite.config.ts → 加入 @vitejs/plugin-react
tsconfig.json → 基础配置 (strict: false)
src/main.tsx → React 入口
src/App.tsx → 路由定义

步骤 3: 验证
npm run dev → 能打开 React 空页面
npm run build → 构建成功
python3 build.py → 部署正常

测试:
tests/test_frontend_build.py → 验证构建不报错
手动打开浏览器确认首页渲染
```

Phase 1: Monitor 页面 + StockCard 组件（首期迁移）

```
步骤 1: 类型定义
src/types/index.ts → API 返回数据接口
```

步骤 2: API 封装
src/api/index.ts → fetch 封装 (所有 API 端点)
src/hooks/useApi.ts → useData / useMutate hook

步骤 3: StockCard 组件
src/components/StockCard.tsx → 从 stock_card.js 翻译
src/components/StockCard.css → CSS Modules

步骤 4: Monitor 页面
src/pages/Monitor.tsx → 从 monitor.html + monitor.js 翻译
server.py 新增 /monitor 路由 → React 版 HTML

步骤 5: 验证
旧 /monitor.html (旧版) 和 /monitor (React 版) 同时可访问
数据一致

测试:
tests/test_monitor_page.py → API 响应结构验证
手动对比新旧两个版本的数据一致性

...后续 Phase 2-4 逐步推进，每页独立提交。

8. 待讨论的问题

#	问题	状态
1	首期从 Monitor（盯盘）页面开始迁移，可以吗？	☒ 待聊
2	SPA（单页）还是保留多页但每页独立 React 实例？	☒ 待聊
3	旧页面的 URL 路径怎么处理？ /monitor 给新版， /monitor-old.html 保留旧版？	☒ 待聊
4	CSS 是全部重写还是渐进迁移（先用 CSS Modules 包裹现有样式）？	☒ 待聊
5	后端生成的 SVG 图表保持现状（服务器端生成），还是逐步转成前端可交互的 ECharts/Canvas？	☒ 待聊